ToxoAI — Email Verification Authentication Flow

How registration, email confirmation, login, and authenticated requests work in the Toxo_AI_code project, and which frontend/backend files are involved.

1. Big picture

The app is a single-page frontend (plain HTML/CSS/JS) that talks to a FastAPI backend over a JSON REST API. In production, nginx serves the frontend and proxies every `/api/` request to the FastAPI process running on port 8000. Authentication is JWT-based: after login the frontend stores a token in `localStorage` and sends it as a `Bearer` header on every protected request.

Project URLs

Purpose	Production	Local development
Frontend (web app)	https://mychatbotproject.uk	http://localhost:8080
Backend API base	https://mychatbotproject.uk/api/v1	http://localhost:8000/api/v1
Health check	https://mychatbotproject.uk/health	http://localhost:8000/health

Auth endpoints (all under `/api/v1/auth`)

Method & path	Who calls it	What it does
`POST /register`	Frontend register form	Creates an unverified user, emails a confirmation link
`GET /verify-email?token=...`	Link inside the email	Marks the user verified, redirects to the frontend
`POST /login`	Frontend login form	Checks password + verification status, returns a JWT
`GET /me`	Frontend after login	Returns the logged-in user's profile (needs Bearer token)

2. Step-by-step flow

Step 1 — Registration

- User fills the `#register-form` in `frontend_files/index.html` (username, email, password) and submits.
- `frontend_files/app.js` → `handleRegister()` sends `POST {API}/api/v1/auth/register` with `{ username, email, password }`.
- `backend_files/main.py` → `register()`:
  - • Rejects if a **verified** user already has that username or email (400 Bad Request).
  - • Deletes any leftover **unverified** row with the same username/email (so a typo'd email or an abandoned signup doesn't block a retry).
  - • Hashes the password with bcrypt (`backend_files/auth.py` → `get_password_hash()`) and creates a new `User` row with `is_verified=False` plus a random `verification_token` (`secrets.token_urlsafe(32)`) that expires in `VERIFICATION_TOKEN_EXPIRE_HOURS` (default 24h).
  - • Calls `backend_files/emails.py` → `send_verification_email()`, which uses the **Resend** API to email a confirmation link. If sending fails, the new user row is rolled back (no account is left half-created).
  - • Responds `201` with `{"message": "Account created. Check your email to confirm it before signing in."}`. The frontend displays this message.

Step 2 — Email confirmation

- The email (sent via Resend, configured by `RESEND_API_KEY` / `EMAIL_FROM`) contains a link: `{BACKEND_URL}/api/v1/auth/verify-email?token=<token>`.
- Clicking it sends a `GET` request straight to the backend — `backend_files/main.py` → `verify_email()`.
- The backend looks up the user by `verification_token`. If the token doesn't exist or is expired → `400 Invalid or expired verification link`.
- If valid: sets `is_verified=True`, clears the token and its expiry, commits to the database.
- Redirects (HTTP 307) the browser to `{FRONTEND_URL}/?verified=true` (or `=false` on failure — though failures actually return a 400 directly instead of a redirect, see note below).
- Back on the frontend, `app.js` → `handleEmailVerificationRedirect()` (run on page load) reads `?verified=...`, shows a toast ("Email confirmed! You can now sign in."), switches to the login tab, and strips the query parameter from the URL.

Step 3 — Login

- User fills the `#login-form` (email + password). `app.js` → `handleLogin()` sends `POST {API}/api/v1/auth/login`.
- `backend_files/main.py` → `login()` looks the user up by email and verifies the password with `verify_password()` (bcrypt).
- If `is_verified` is still `False` → `403 Please confirm your email before signing in.` This is the gate that enforces "no usable account without a real, reachable email".
- If `is_active` is `False` → `400 Inactive user`.
- Otherwise, `auth.py` → `create_access_token()` signs a JWT (HS256, secret = `SECRET_KEY`) with `sub=username` and a 60-minute expiry (`ACCESS_TOKEN_EXPIRE_MINUTES`). Returned as `{ access_token, token_type: "bearer" }`.
- The frontend stores it with `setToken()` (`localStorage`) and calls `loadApp()`.

Step 4 — Authenticated requests

- Every protected call (e.g. `GET /me`, `/documents`, `/chat`) goes through `app.js` → `authHeaders()`, which adds `Authorization: Bearer <token>`.
- On the backend, the `get_current_user` dependency (`main.py`) calls `auth.verify_token()` to decode/validate the JWT, extracts the username from `sub`, and loads the matching `User` row. An invalid/expired token or unknown user → `401 Unauthorized`, which the frontend treats as "session expired" and sends the user back to the auth page.

Note on re-registration: if a user registers again with the same username/email while still unverified (e.g. they mistyped the address or lost the email), the old pending row is deleted and a fresh row + token + email are generated — effectively a built-in "resend confirmation".

3. Files involved

Backend — `backend_files/`

File	Role in authentication
`main.py`	Defines the FastAPI app and the /api/v1/auth routes: register, verify-email, login, me. Also the get_current_user dep
`auth.py`	JWT creation/verification (create_access_token, verify_token), password hashing (passlib/bcrypt), Pydantic schema
`models.py`	SQLAlchemy User model: id, username, email, hashed_password, is_active, plus the new is_verified, verification_to
`database.py`	SQLite engine/session setup. init_db() creates tables and now also ALTER TABLEs existing DBs to add the new ve

`emails.py`	New file. Configures the Resend client (RESEND_API_KEY, EMAIL_FROM, BACKEND_URL, FRONTEND_URL) a

Frontend — `frontend_files/`

File	Role in authentication
`index.html`	Auth page markup: the #login-form and #register-form, tab switching between them, and the app shell shown after lo
`app.js`	API base URLs (AUTH_API = .../api/v1/auth). Implements handleRegister(), handleLogin(), token storage helpers (g
`style.css`	Visual styling for the auth card and forms (no auth logic).

4. Flow diagram

```
Browser (frontend_files)             FastAPI backend (backend_files)        Resend
----------------------               --------------------------------     -------

[Register form] ---POST /api/v1/auth/register---> main.py: register()
                                          - validates uniqueness (verified only)
                                          - creates User(is_verified=False)
                                          - generates verification_token
                                          - emails.send_verification_email() ----> sends email
                <--- 201 "check your email" ---

(user opens inbox, clicks link)
GET {BACKEND_URL}/api/v1/auth/verify-email?token=... ---> main.py: verify_email()
                                          - finds user by token, checks expiry
                                          - sets is_verified=True
                <--- 307 redirect to {FRONTEND_URL}/?verified=true ---

[app.js on load] reads ?verified=true
-> toast "Email confirmed!" + switch to Sign In tab

[Login form] ---POST /api/v1/auth/login---> main.py: login()
                                          - checks password
                                          - checks is_verified (403 if not)
                                          - issues JWT (sub=username, 60 min)
                <--- 200 { access_token } ---

localStorage.token = access_token

[Any protected call] ---GET /api/v1/auth/me, Authorization: Bearer <jwt>--->
                                         get_current_user() decodes JWT -> User
                <--- 200 user profile ---
```

5. Configuration (environment variables)

Variable	Default	Purpose
`SECRET_KEY`	(insecure placeholder)	Signs/verifies JWT access tokens (auth.py)
`RESEND_API_KEY`	— (required)	Auth for the Resend email API (emails.py)
`EMAIL_FROM`	ToxoAI <onboarding@resend.dev>	"From" address of confirmation emails
`VERIFICATION_TOKEN_EXPIRE_HOURS`	24	How long a confirmation link stays valid
`BACKEND_URL`	http://localhost:8000	Used to build the verify-email link (prod: https://mychatbotproj
`FRONTEND_URL`	http://localhost:8080	Where the user lands after clicking the link (prod: https://mych

These are set in the server environment (e.g. systemd unit / docker-compose), documented in `.env.example`. They are not loaded from a `.env` file automatically — only os.environ is read.

Why double opt-in satisfies "the email must exist": the account row is created with is_verified=False and login is blocked until the confirmation link is clicked. If the address doesn't exist (or the user never receives/clicks it), the account simply never becomes usable — no extra deliverability-checking API is needed.